

ITF23019: Parallel and Distributed Programming

Spring Semester, 2026

Lab13: MapReduce

Submission Deadline: April 29th, 2026 23:59

You need at least 50 points to pass this lab.

Note: You may experience that your virtual machine does not response quickly with your commands. In addition, may get some errors messages while doing this lab but it is not abnormal.

1 What is MapReduce?

MapReduce is a programming model to process very large datasets in distributed environments based on Java. It is said to be a used for distributed computing and for **BigData** analysis as the model provides capability for processing the huge volumes of complex data. Therefore, MapReduce is used to write applications to process the data in parallel on multiple nodes or many machines working in a cluster in a reliable maner. **MapReduce** model consists of two main functions: the **Map** function and the **Reduce** function (and hence the name).

- The **Map** function takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key-value pairs). **Map** is executed in multiple machines (hence the distributed systems) at the same time (hence parallelism).

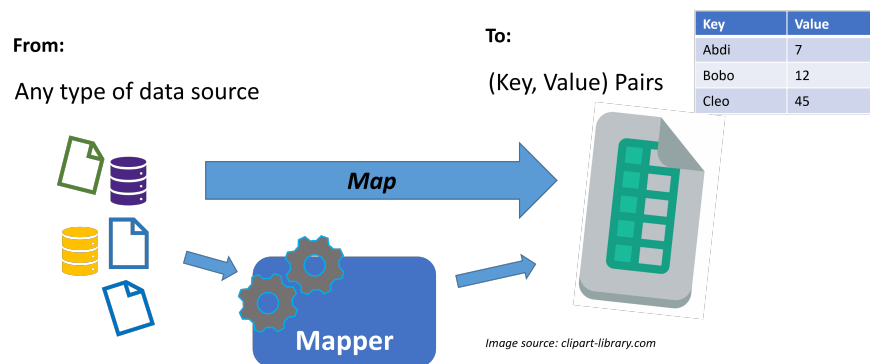


Figure 1: Map function.

- The **Reduce** function takes the output from **Map** function as its input and then reduces its size by performing some kinds of termination operations on the dataset. In particular, it combines those data tuples (key-value pairs) into a smaller set of tuples.

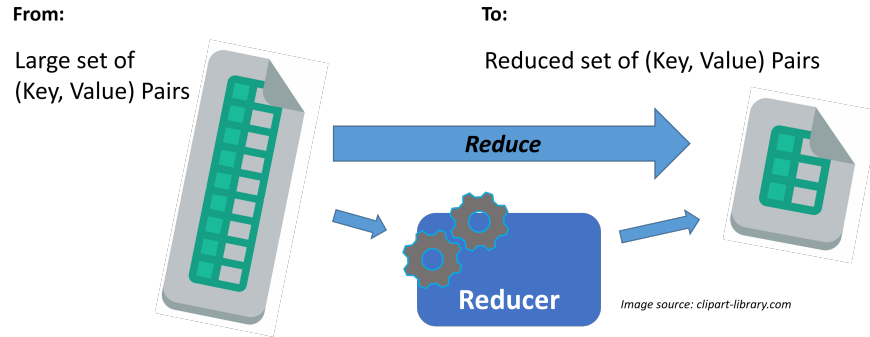


Figure 2: Reduce function.

Under the MapReduce model, the data processing primitives are called **mappers** and **reducers**. The data is **Mapped** by a **mapper** and **Reduced** by a **reducer**. Decomposing a data processing application into mappers and reducers is sometimes nontrivial. Therefore, the data is mapped by a Mapper and Reduced by a Reducer.

MapReduce speeds up Big data processing by splitting up the big dataset into smaller datasets and then processes them in parallel. The major advantage of MapReduce is that it is easy to scale data

Map + Reduce = MapReduce

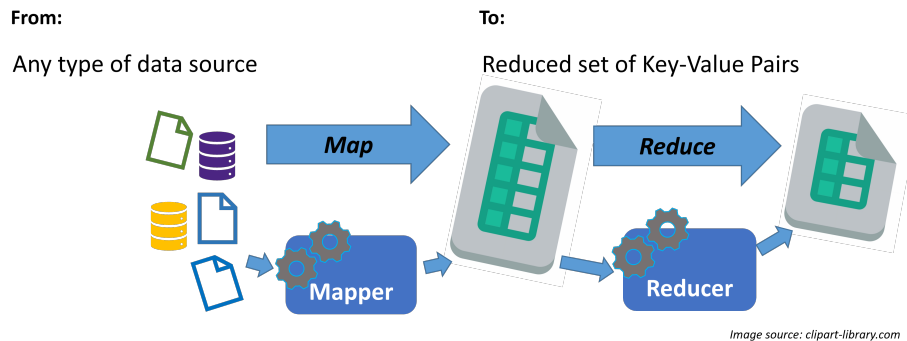


Figure 3: Map + Reduce = MapReduce.

processing over multiple computing nodes. Once we write an application in the MapReduce form, scaling the application to run over hundreds, thousands, or even tens of thousands of machines in a cluster is merely a configuration change. This simple scalability is what has attracted many programmers to use the MapReduce model [2,3].

2 How MapReduce Works?

Figure 1 shows typical phases in MapReduce form in which **Map** and **Reduce** are the main phases.

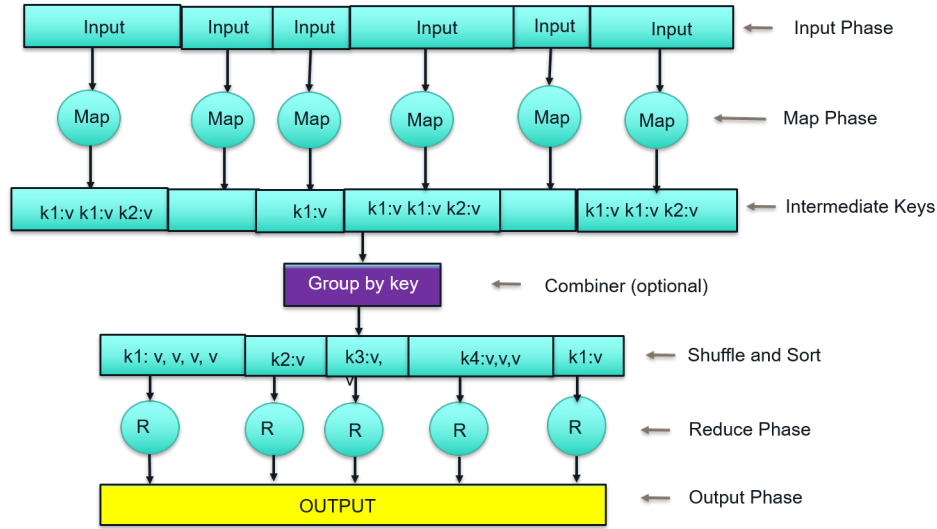


Figure 4: WDiffrent phases in MapReduce Model[2].

- **Input Phase:** Here we have a Record Reader that translates each record in an input file and sends the parsed data to the mapper in the form of key-value pairs.
- **Map:** Map is a user-defined function, which takes a series of key-value pairs and processes each one of them to generate zero or more key-value pairs
- **Intermediate Keys:** They key-value pairs generated by the mapper are known as intermediate keys.
- **Combiner:** A combiner is a type of local Reducer that groups similar data from the map phase into identifiable sets. It takes the intermediate keys from the mapper as input and applies a user-defined code to aggregate the values in a small scope of one mapper. It is not a part of the main MapReduce algorithm; it is optional.
- **Shuffle and Sort:** The Reducer task starts with the Shuffle and Sort step. It downloads the grouped key-value pairs onto the local machine, where the Reducer is running. The individual key-value pairs are sorted by key into a larger data list. The data list groups the equivalent keys together so that their values can be iterated easily in the Reducer task.
- **Reducer:** The Reducer takes the grouped key-value paired data as input and runs a Reducer function on each one of them. Here, the data can be aggregated, filtered, and combined in a number of ways, and it requires a wide range of processing. Once the execution is over, it gives zero or more key-value pairs to the final step.
- **Output Phase:** In the output phase, we have an output formatter that translates the final key-value pairs from the Reducer function and writes them onto a file using a record writer.

Figure 2 is one explanation of the above phases. The input dataset consisting of alphabet. The first step is to split this input dataset into smaller datasets and distribute the datasets to multiple nodes to process simultaneously. In the Map phase, the data in each node is converted into (key, value) pairs. The **Map** function counts the number of repetitions for each letter. Here, the letter is considered as the **key** and the number is considered as the **value**. For example, the first splitted dataset [A B R] is converted into (key, value) pairs which are (A, 1), (B,1), and (R,1). Here, A, B and R are key and 1 is the value. The **Shuffle**

and **Sort** phase sorts new data by keys with alphabet order. Finally, the **Reduce** function reduces data size by the keys with sum operation.

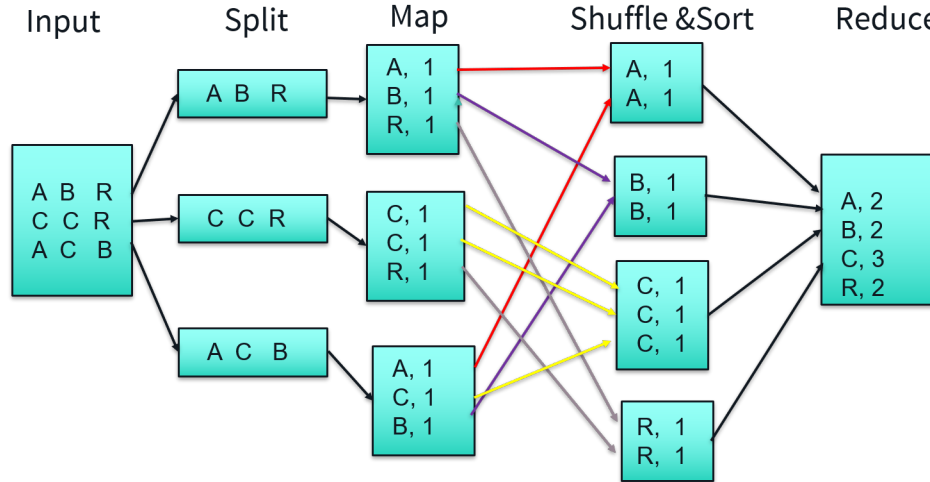


Figure 5: One example of **MapReduce** model.

3 Running a MapReduce program

In this lab, we will run a MapReduce Program in Hadoop Distributed File System (HDFS). HDFS is a special purpose file system that is designed to work with the MapReduce engine. It provides high-throughput access to data in a highly distributed environment. You may need to refer to some steps in previous lab.

- Run Docker Desktop.
- Run two containers, start with `sandbox-proxy` and then `sandbox-hdp`. At this step, pay attention to CPUs and memory are usage. For example:

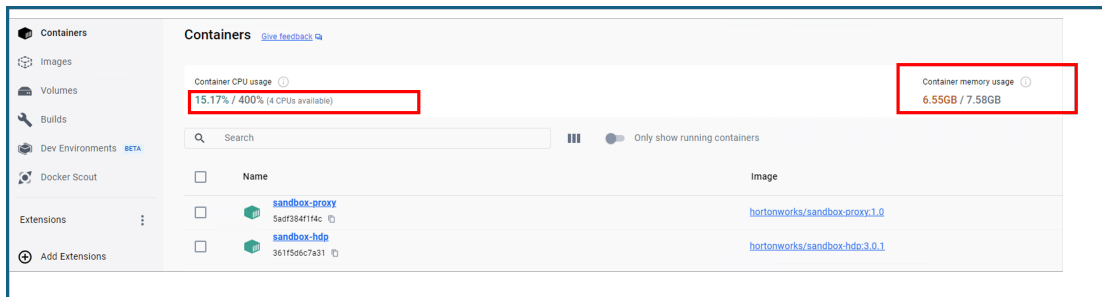


Figure 6: Docker with two containers.

- Navigate to `localhost:8080` or `127.0.0.1:8080`. Then, login with **admin** account (`admin/ITF23019`).

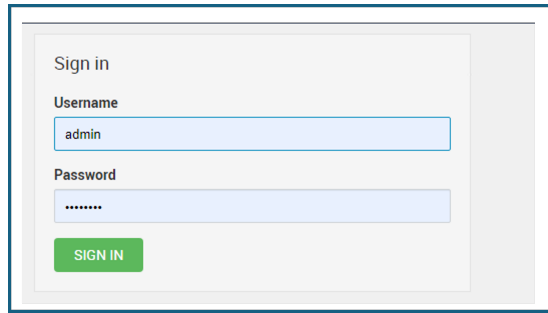


Figure 7: Login Ambari with admin account.

It is time to be careful now. To the left in Ambari, we see that there are many services - in Figure 8.

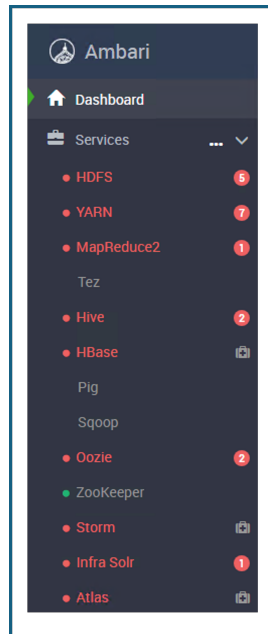


Figure 8: Services in Ambari.

If all services are running, CPUs will be overloaded shortly. As a result, your VM will respond very slowly with your actions later on. Therefore, it is better to check whether Ambari runs all services automatically or not. Click on the symbol in the red box in Figure 28 to check background operations in Ambari:

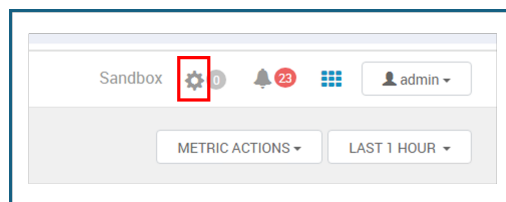


Figure 9: Check Background Operations in Ambari (1/2).

If you see that **Start All Services** operation is running, like the figure below:

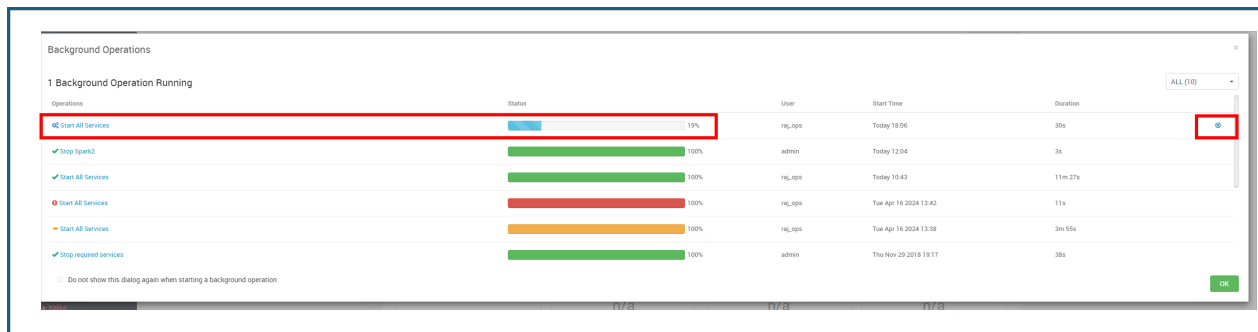


Figure 10: Check Background Operations in Ambari (2/2).

You need to abort **Start All Services** operation. Click on the symbol in the small rectangular in Figure 10 - similar to the one in Figure 11

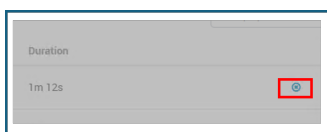


Figure 11: Stop **Start All Services** operation in Ambari.

Then, click on OK button to abort **Start All Services** operation:

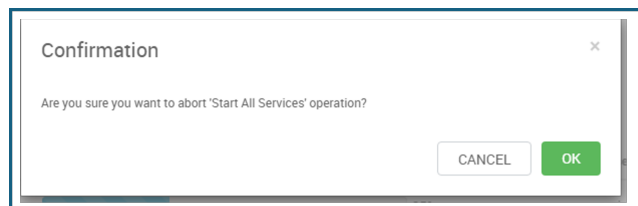


Figure 12: **Start All Services** operation.

Even we do not need all services to run, we do need the following services: **ZooKeeper**, **HDFS**, **MapReduce2** and **YARN** are started before running this part. Start these services in sequence. See the following figures:

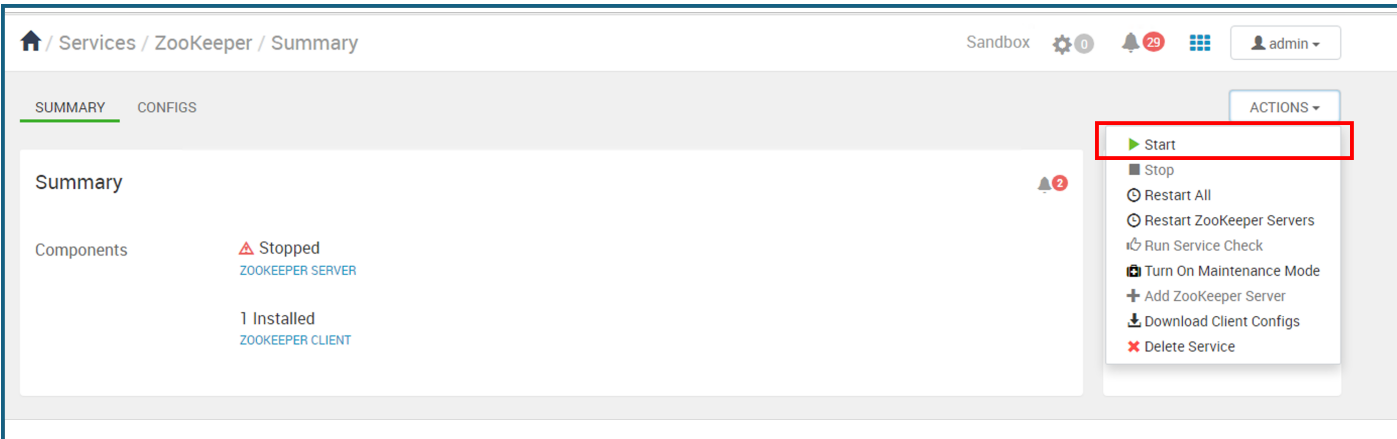


Figure 13: Start **ZooKeeper** service.

Once the service is started successfully, it will look like this:

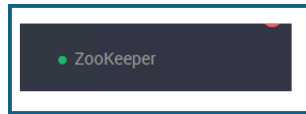


Figure 14: **ZooKeeper** service is fully started.

- Do similarly with **HDFS**, **MapReduce2**, **YARN**.

When the status of the four services turns green, it is ready for other operations.

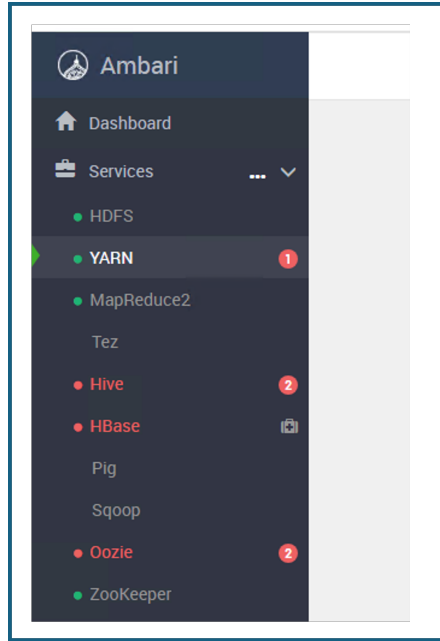


Figure 15: **ZooKeeper, HDFS, MapReduce2 and YARN** are ready.

Before executing a MapReduce program, let's start some simple File System Operations in HDFS:

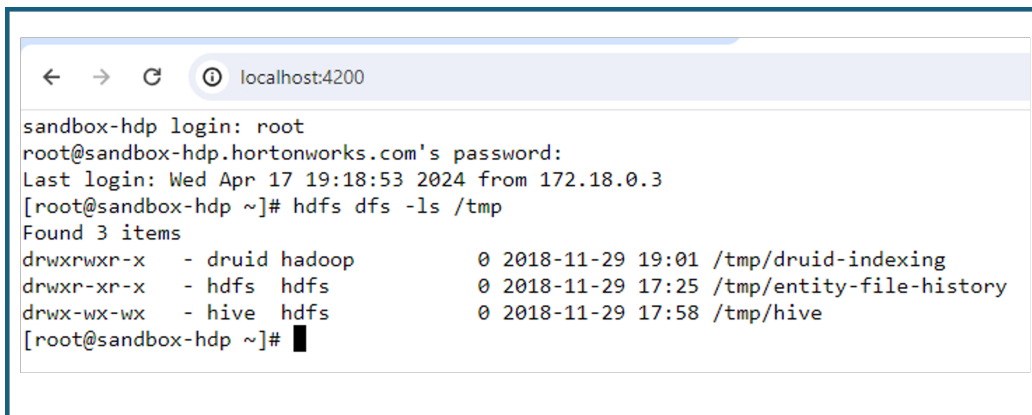
3.1 Practice File System Operations: To/From HDFS

HDFS is the primary storage system used by Hadoop applications. It rapidly transfers data between nodes. HDFS is a key component of many Hadoop systems, as it provides a means for managing big data, as well as supporting big data analytics. More information about HDFS can be found in [3].

We will start with **creating** and **deleting** HDFS folders. We will be using `hdfs` commands for this purpose. Similar to many commands in Linux, we will use `ls` to list files in a directory, `-mkdir` to create directories, `-rmdir` to remove a directory and so on.

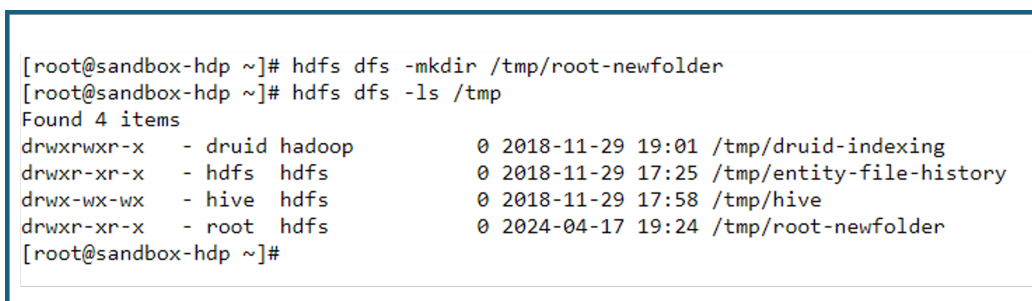
There are quite several accounts available in Ambari. However, account management is not the focus of this lab. It is good enough to just use them. We will perform some simple operations with temporary (tmp) directories for some accounts,

- **root** account (**root/ITF23019**)

A terminal window with a blue header bar showing navigation icons and the address 'localhost:4200'. The terminal text shows a login for 'root' at 'sandbox-hdp.hortonworks.com', followed by a password prompt and a successful login message. The user then runs 'hdfs dfs -ls /tmp', which lists three items: 'druid-indexing' (owned by druid/hadoop), 'entity-file-history' (owned by hdfs/hdfs), and 'hive' (owned by hive/hdfs).

```
sandbox-hdp login: root
root@sandbox-hdp.hortonworks.com's password:
Last login: Wed Apr 17 19:18:53 2024 from 172.18.0.3
[root@sandbox-hdp ~]# hdfs dfs -ls /tmp
Found 3 items
drwxrwxr-x - druid hadoop      0 2018-11-29 19:01 /tmp/druid-indexing
drwxr-xr-x - hdfs hdfs        0 2018-11-29 17:25 /tmp/entity-file-history
drwx-wx-wx - hive hdfs        0 2018-11-29 17:58 /tmp/hive
[root@sandbox-hdp ~]#
```

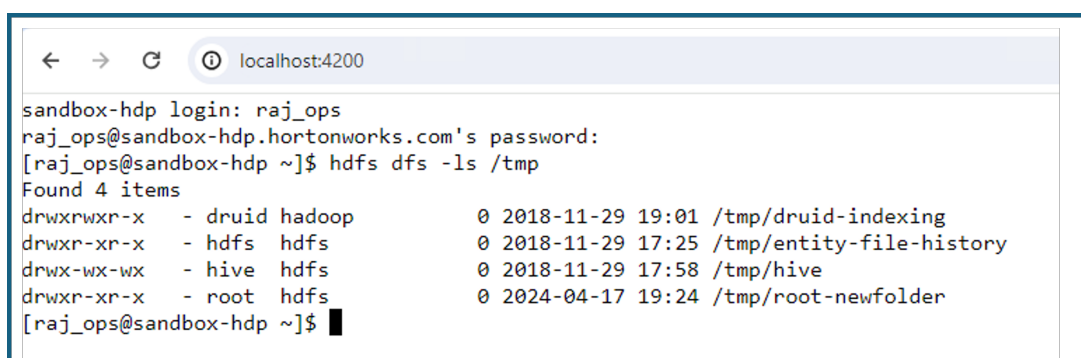
Figure 16: Some HDFS folder operations with root account (1/2).

A terminal window showing the same root user session as Figure 16. The user runs 'hdfs dfs -mkdir /tmp/root-newfolder', and then 'hdfs dfs -ls /tmp'. The output now lists four items, including the newly created 'root-newfolder' owned by root/hdfs.

```
[root@sandbox-hdp ~]# hdfs dfs -mkdir /tmp/root-newfolder
[root@sandbox-hdp ~]# hdfs dfs -ls /tmp
Found 4 items
drwxrwxr-x - druid hadoop      0 2018-11-29 19:01 /tmp/druid-indexing
drwxr-xr-x - hdfs hdfs        0 2018-11-29 17:25 /tmp/entity-file-history
drwx-wx-wx - hive hdfs        0 2018-11-29 17:58 /tmp/hive
drwxr-xr-x - root hdfs        0 2024-04-17 19:24 /tmp/root-newfolder
[root@sandbox-hdp ~]#
```

Figure 17: Some HDFS folder operations with root account (2/2).

- raj_ops account (same user and password)

A terminal window showing a login for 'raj_ops' at 'sandbox-hdp.hortonworks.com'. The user runs 'hdfs dfs -ls /tmp', which lists the same four items as in Figure 17, including 'root-newfolder' owned by root/hdfs.

```
sandbox-hdp login: raj_ops
raj_ops@sandbox-hdp.hortonworks.com's password:
[raj_ops@sandbox-hdp ~]$ hdfs dfs -ls /tmp
Found 4 items
drwxrwxr-x - druid hadoop      0 2018-11-29 19:01 /tmp/druid-indexing
drwxr-xr-x - hdfs hdfs        0 2018-11-29 17:25 /tmp/entity-file-history
drwx-wx-wx - hive hdfs        0 2018-11-29 17:58 /tmp/hive
drwxr-xr-x - root hdfs        0 2024-04-17 19:24 /tmp/root-newfolder
[raj_ops@sandbox-hdp ~]$
```

Figure 18: Some HDFS folder operations with raj_ops account.

```

[raj_ops@sandbox-hdp ~]$ hdfs dfs -mkdir /tmp/raj_ops-newfolder
[raj_ops@sandbox-hdp ~]$ hdfs dfs -ls /tmp
Found 5 items
drwxrwxr-x - druid   hadoop      0 2018-11-29 19:01 /tmp/druid-indexing
drwxr-xr-x - hdfs    hdfs        0 2018-11-29 17:25 /tmp/entity-file-history
drwx-wx-wx - hive    hdfs        0 2018-11-29 17:58 /tmp/hive
drwxr-xr-x - raj_ops hdfs        0 2024-04-17 19:29 /tmp/raj_ops-newfolder
drwxr-xr-x - root    hdfs        0 2024-04-17 19:24 /tmp/root-newfolder
[raj_ops@sandbox-hdp ~]$ █

```

Figure 19: Some HDFS folder operations with `raj_ops` account.

- `maria_dev` account (same user and password)

We will do a number of operations with this account. Some of which are: 1) **Create** a simple file at a known location as previous lab; 2) **Copy** a new file to a place in the HDFS file system using the `hdfs dfs -put` command and 3) **Verify** the created file using `hdfs dfs -ls` command and 4) **Check** its content with `hdfs dfs -cat` command:

```

localhost:4200
sandbox-hdp login: maria_dev
maria_dev@sandbox-hdp.hortonworks.com's password:
Last login: Wed Apr 17 18:56:28 2024 from 172.18.0.3
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /tmp
Found 5 items
drwxrwxr-x - druid   hadoop      0 2018-11-29 19:01 /tmp/druid-indexing
drwxr-xr-x - hdfs    hdfs        0 2018-11-29 17:25 /tmp/entity-file-history
drwx-wx-wx - hive    hdfs        0 2018-11-29 17:58 /tmp/hive
drwxr-xr-x - raj_ops hdfs        0 2024-04-17 19:29 /tmp/raj_ops-newfolder
drwxr-xr-x - root    hdfs        0 2024-04-17 19:24 /tmp/root-newfolder
[maria_dev@sandbox-hdp ~]$ █

```

Figure 20: Some HDFS folder operations with `maria_dev` account (1/3).

```
← → ↻ ⓘ localhost:4200
sandbox-hdp login: maria_dev
maria_dev@sandbox-hdp.hortonworks.com's password:
[maria_dev@sandbox-hdp ~]$ mkdir ITF23019
[maria_dev@sandbox-hdp ~]$ ls
ITF23019
[maria_dev@sandbox-hdp ~]$ cd ITF23019
[maria_dev@sandbox-hdp ITF23019]$ mkdir new_folder
[maria_dev@sandbox-hdp ITF23019]$ cd new_folder/
[maria_dev@sandbox-hdp new_folder]$ cat >linux-file.txt
This file is created in Linux.
^X^Z
[1]+  Stopped                  cat > linux-file.txt
[maria_dev@sandbox-hdp new_folder]$ cd
[maria_dev@sandbox-hdp ~]$ hdfs dfs -mkdir /tmp/maria_dev-newfolder
[maria_dev@sandbox-hdp ~]$
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /tmp
Found 6 items
drwxrwxr-x - druid      hadoop          0 2018-11-29 19:01 /tmp/druid-indexing
drwxr-xr-x - hdfs       hdfs           0 2018-11-29 17:25 /tmp/entity-file-history
drwx-wx-wx - hive       hdfs           0 2018-11-29 17:58 /tmp/hive
drwxr-xr-x - maria_dev hdfs           0 2024-04-12 11:50 /tmp/maria_dev-newfolder
drwxr-xr-x - raj_ops   hdfs           0 2024-04-12 11:44 /tmp/raj_ops-newfolder
drwxr-xr-x - root      hdfs           0 2024-04-12 11:40 /tmp/root-newfolder
[maria_dev@sandbox-hdp ~]$ hdfs dfs -put ~/ITF23019/new_folder/linux-file.txt /tmp/maria_dev-newfolder
[maria_dev@sandbox-hdp ~]$ hdfs dfs -cat /tmp/maria_dev-newfolder
cat: `/tmp/maria_dev-newfolder': Is a directory
[maria_dev@sandbox-hdp ~]$ hdfs dfs -cat /tmp/maria_dev-newfolder/linux-file.txt
This file is created in Linux.
[maria_dev@sandbox-hdp ~]$
```

Figure 21: Some HDFS folder operations with maria.dev account (2/3).

```
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /tmp/maria_dev-newfolder
Found 1 items
-rw-r--r-- 1 maria_dev hdfs      31 2024-04-12 12:05 /tmp/maria_dev-newfolder/linux-file.txt
[maria_dev@sandbox-hdp ~]$ hdfs dfs -cp /tmp/maria_dev-newfolder/linux-file.txt /tmp/maria_dev-newfolder/linux-file-copy.txt
[maria_dev@sandbox-hdp ~]$ hdfs dfs -ls /tmp/maria_dev-newfolder
Found 2 items
-rw-r--r-- 1 maria_dev hdfs      31 2024-04-12 12:09 /tmp/maria_dev-newfolder/linux-file-copy.txt
-rw-r--r-- 1 maria_dev hdfs      31 2024-04-12 12:05 /tmp/maria_dev-newfolder/linux-file.txt
[maria_dev@sandbox-hdp ~]$ hdfs dfs -cat /tmp/maria_dev-newfolder/linux-file-copy.txt
This file is created in Linux.
[maria_dev@sandbox-hdp ~]$ hdfs dfs -get /tmp/maria_dev-newfolder/linux-file-copy.txt
[maria_dev@sandbox-hdp ~]$ ls -la
total 32
drwx----- 1 maria_dev maria_dev 4096 Apr 12 12:11 .
drwxr-xr-x 1 root      root      4096 Nov 29  2018 ..
-rw-r--r-- 1 maria_dev maria_dev  18 Apr 11  2018 .bash_logout
-rw-r--r-- 1 maria_dev maria_dev 193 Apr 11  2018 .bash_profile
-rw-r--r-- 1 maria_dev maria_dev 619 Nov 29  2018 .bashrc
drwxrwxr-x 3 maria_dev maria_dev 4096 Apr 12 11:48 ITF23019
-rw-r--r-- 1 maria_dev maria_dev  31 Apr 12 12:11 linux-file-copy.txt
[maria_dev@sandbox-hdp ~]$
```

Figure 22: Some HDFS folder operations with maria_dev account (3/3).

3.2 Running a MapReduce program

- Login with maria_dev account and make a new folder named MapReduce

```
sandbox-hdp login: maria_dev
maria_dev@sandbox-hdp.hortonworks.com's password:
Last login: Fri Apr 12 20:18:48 2024 from 172.18.0.2
[maria_dev@sandbox-hdp ~]$ ls
[maria_dev@sandbox-hdp ~]$ mkdir MapReduce
[maria_dev@sandbox-hdp ~]$ cd MapReduce/
```

Figure 23: Some HDFS folder operations with raj_ops account.

- `git clone https://github.com/thuynga20/WordCount_Hadoop.git`

```
[maria_dev@sandbox-hdp MapReduce]$ git clone https://github.com/thuynga20/WordCount_Hadoop.git
Cloning into 'WordCount_Hadoop'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (15/15), done.
remote: Total 18 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (18/18), done.
[maria_dev@sandbox-hdp MapReduce]$
```

Figure 24: Clone WordCount.java to maria_dev user.

- Setup environment variables


```
export JAVA_HOME=/usr/java/default
export PATH=${JAVA_HOME}/bin:${PATH}
export HADOOP_CLASSPATH=${JAVA_HOME}/lib/tools.jar
```

- Compile the WordCount.java file:

```
hadoop com.sun.tools.javac.Main WordCount.java
```

- Create a .jar file:

```
jar cf wc.jar WordCount*.class
```

```
WordCount_Hadoop
[maria_dev@sandbox-hdp MapReduce]$ export JAVA_HOME=/usr/java/default
[maria_dev@sandbox-hdp MapReduce]$ export PATH=${JAVA_HOME}/bin:${PATH}
[maria_dev@sandbox-hdp MapReduce]$ export HADOOP_CLASSPATH=${JAVA_HOME}/lib/tools.jar
[maria_dev@sandbox-hdp MapReduce]$ cd WordCount_Hadoop/
[maria_dev@sandbox-hdp WordCount_Hadoop]$ ls
MapReduceInput  readme.md  WordCount.java
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hadoop com.sun.tools.javac.Main WordCount.java
[maria_dev@sandbox-hdp WordCount_Hadoop]$ jar cf wc.jar WordCount*.class
[maria_dev@sandbox-hdp WordCount_Hadoop]$ ls
MapReduceInput  readme.md  wc.jar  WordCount.class  WordCount$IntSumReducer.class  WordCount.java  WordCount$TokenizerMapper.class
[maria_dev@sandbox-hdp WordCount_Hadoop]$
```

Figure 25: Setting environment variables and compiling java file.

- Execute The MapReduce job with the Hadoop command:

```
hadoop jar wc.jar WordCount input_path output_path
```

Therefore, you need to create `input_path`

```
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hdfs dfs -ls /tmp/ITF23019
Found 1 items
drwxr-xr-x - maria_dev hdfs      0 2024-04-12 20:40 /tmp/ITF23019/input
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hdfs dfs -put input.txt /tmp/ITF23019/input
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hdfs dfs -ls /tmp/ITF23019/input
Found 1 items
-rw-r--r-- 1 maria_dev hdfs      71 2024-04-12 20:43 /tmp/ITF23019/input/input.txt
[maria_dev@sandbox-hdp WordCount_Hadoop]$
```

Figure 26: Create an input file for a WordCount.java program.

```
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hadoop jar wc.jar WordCount /tmp/ITF23019/input/input.txt /tmp/ITF23019/lab13_output
```

Figure 27: Running the WordCount.java program with input and output directories.

Note that you may get the error message saying that the `input.txt` file does not exit. If that happens, just create a `input.txt` file by using `cat >input.txt` command and type some text in it. After that, using `Ctrl+D` to stop editing the `input.txt` file. If the command in Figure 27 runs correctly, you will see something like:

```
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hadoop jar wc.jar WordCount /tmp/ITF23019/input/input.txt /tmp/ITF23019/lab13_output
25/04/08 19:32:34 INFO client.RMProxy: Connecting to ResourceManager at sandbox-hdp.hortonworks.com/172.18.0.2:8050
25/04/08 19:32:34 INFO client.AHSProxy: Connecting to Application History server at sandbox-hdp.hortonworks.com/172.18.0.2:10200
25/04/08 19:32:34 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute
25/04/08 19:32:34 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /user/maria_dev/.staging/job_1744133152645_0004
25/04/08 19:32:35 INFO input.FileInputFormat: Total input files to process : 1
25/04/08 19:32:35 INFO mapreduce.JobSubmitter: number of splits:1
25/04/08 19:32:35 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1744133152645_0004
25/04/08 19:32:35 INFO mapreduce.JobSubmitter: Executing with tokens: []
25/04/08 19:32:36 INFO conf.Configuration: found resource resource-types.xml at file:/etc/hadoop/3.0.1.0-187/0/resource-types.xml
25/04/08 19:32:36 INFO impl.YarnClientImpl: Submitted application application_1744133152645_0004
25/04/08 19:32:36 INFO mapreduce.Job: The url to track the job: http://sandbox-hdp.hortonworks.com:8088/proxy/application_1744133152645_0004/
25/04/08 19:32:36 INFO mapreduce.Job: Running job: job_1744133152645_0004
25/04/08 19:32:43 INFO mapreduce.Job: Job job_1744133152645_0004 running in uber mode : false
25/04/08 19:32:43 INFO mapreduce.Job: map 0% reduce 0%
25/04/08 19:32:49 INFO mapreduce.Job: map 100% reduce 0%
25/04/08 19:32:55 INFO mapreduce.Job: map 100% reduce 100%
25/04/08 19:32:55 INFO mapreduce.Job: Job job_1744133152645_0004 completed successfully
25/04/08 19:32:55 INFO mapreduce.Job: Counters: 53
```

Figure 28: Expected output.

The file containing the output of the program is stored in file `part-r-000000`, as shown in Figure 29. You can use `cat` command to view the content of the file.

```
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hdfs dfs -ls /tmp/ITF23019/lab13_output
Found 2 items
-rw-r--r-- 1 maria_dev hdfs 0 2025-04-08 19:32 /tmp/ITF23019/lab13_output/_SUCCESS
-rw-r--r-- 1 maria_dev hdfs 59 2025-04-08 19:32 /tmp/ITF23019/lab13_output/part-r-000000
[maria_dev@sandbox-hdp WordCount_Hadoop]$ hdfs dfs -cat /tmp/ITF23019/lab13_output/part-r-000000
Linux 1
This 1
a 1
created 1
file 1
in 1
is 1
new 1
text 1
```

Figure 29: Output of the program.

4 Exercises

4.1 Exercise 1 - 40 points

- Reproduce Figures 16-21. Include screenshots in your report.

4.2 Exercise 2 - 35 points

- Reproduce Figure 29.

Analyse the `WordCount.java` Program

- What does the `main()` function do?
- Which lines perform the Map phase of the MapReduce algorithm?
- Which lines perform the Reduce phase of the MapReduce algorithm?

4.3 Exercise 3 - 25 points

- Where does the input file(s) reside, in the local file system or in the HDFS file system? Why?
- Where is the output file stored, in the local file system or in the HDFS file system? Why?
- Where does your java program reside, in the local file system or in the HDFS file system? Why?

5 What To Submit

Complete the the exercise in this assignment. Put your codes along with your lab13_report file (Markdown, .txt or .pdf file) inside the lab13 directory of your repository. Commit your changes and remember to check the GitHub website to make sure all files have been submitted.

6 Acknowledgement

1. The cost of this lab is covered by our university college.
2. Some parts of this lab are sourced (with agreement) Hogskolan Danarna.

7 References

1. Hadoop - MapReduce. https://www.tutorialspoint.com/hadoop/hadoop_mapreduce.htm.
2. MapReduce - Introduction. https://www.tutorialspoint.com/map_reduce/map_reduce_introduction.htm.
3. [https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs#:~:text=HDFS%20\(Hadoop%20Distributed%20File%20System,handle%20and%20store%20big%20data.](https://www.databricks.com/glossary/hadoop-distributed-file-system-hdfs#:~:text=HDFS%20(Hadoop%20Distributed%20File%20System,handle%20and%20store%20big%20data.)